

# Machine Learning Models

Shaon Khan

## Simple Linear Regression

### ১. মূল সমীকরণ (The Core Formula)

সিম্পল লিনিয়ার রিগ্রেশন মূলত একটি ইনপুট ফিচার (X) এবং একটি আউটপুট টার্গেটের (y) মধ্যে সরলরৈখিক সম্পর্ক তৈরি করে। এর গাণিতিক সূত্রটি হলো:

$$\hat{y} = wX + b$$

$\hat{y}$  (y-hat): মডেলের প্রেডিক্ট করা মান (Predicted Value)

X: ইনপুট ফিচার (Independent Variable)

w (Weight/Slope): রেখাটির ঢাল। X এক ইউনিট বাড়লে y কতটুকু বাড়বে বা কমবে, তা w নির্ধারণ করে

b (Bias/Intercept): ইন্টারসেপ্ট। X-এর মান 0 হলে y-এর মান কত হবে, তা এটি নির্দেশ করে

### ২. কস্ট ফাংশন (Cost Function)

মডেলের প্রেডিকশন আসল ডেটার তুলনায় কতটা ভুল করছে, তা পরিমাপ করার জন্য Mean Squared Error (MSE) কস্ট ফাংশন ব্যবহার করা হয়। এর গাণিতিক রূপ:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

যেহেতু

$$\hat{y}_i = wX_i + b$$

তাই সমীকরণটি দাঁড়ায়:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (wX_i + b - y_i)^2$$

n: মোট ডেটা স্যাম্পলের সংখ্যা

$y_i$ : আসল মান (Actual Value)

$\hat{y}_i$ : মডেলের প্রেডিক্ট করা মান

### ৩. কস্ট ফাংশনের ডেরিভেটিভ (Partial Derivatives)

গ্রাডিয়েন্ট ডিসেন্ট পদ্ধতিতে কস্ট ফাংশন (J) এর মান সর্বনিম্ন করার জন্য আমাদের জানতে হবে w এবং b পরিবর্তনের সাথে সাথে কস্ট কতটা পরিবর্তিত হচ্ছে। এর জন্য ক্যালকুলাসের চেইন রুল (Chain Rule) ব্যবহার করে আংশিক অন্তরীকরণ (Partial Derivative) করা হয়:

w-এর সাপেক্ষে ডেরিভেটিভ (dJ/dw):

$$\frac{\partial J}{\partial w} = \frac{2}{n} \sum_{i=1}^n (w X_i + b - y_i) \cdot X_i$$

b-এর সাপেক্ষে ডেরিভেটিভ (dJ/db):

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum_{i=1}^n (w X_i + b - y_i)$$

(এখানে  $wX_i + b - y_i$  অংশটি হলো মূলত প্রেডিকশন এবং আসল মানের মধ্যকার ভুল বা Error)

### ৪. প্যারামিটার আপডেট সূত্র (Weight & Bias Update)

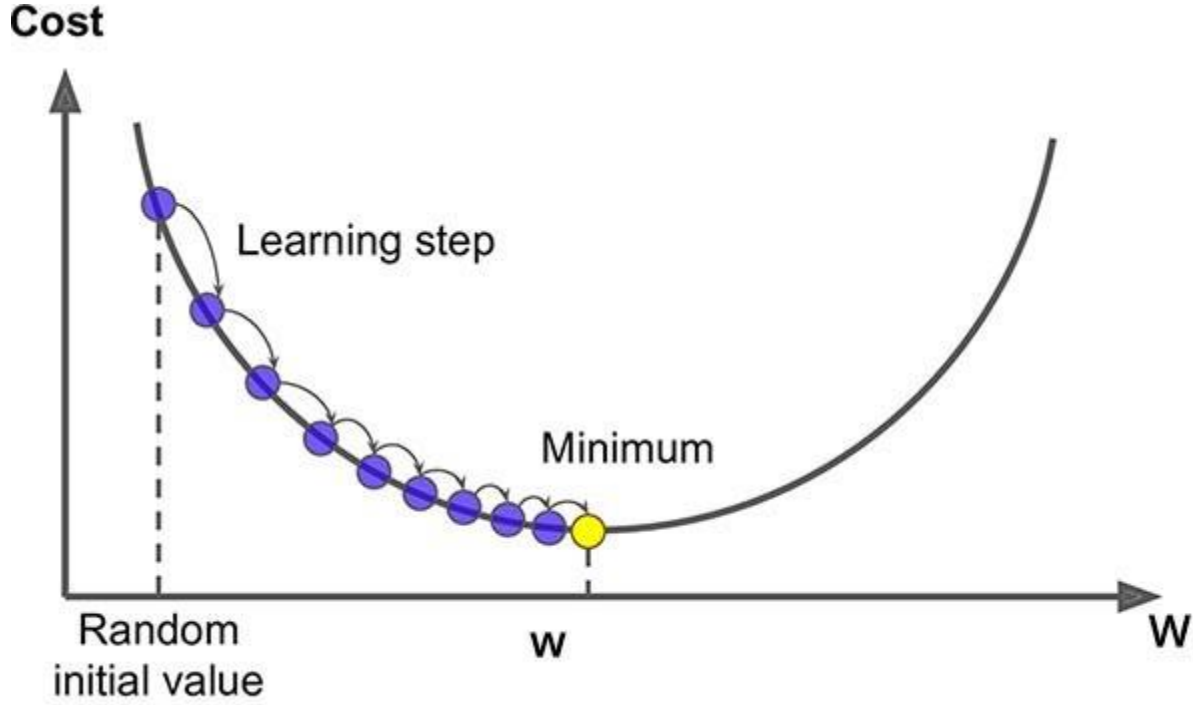
প্রতিটি ইটারেশনে বা লুপে প্রাপ্ত গ্রাডিয়েন্ট বা ডেরিভেটিভের ওপর ভিত্তি করে লার্নিং রেট ( $\alpha$ ) গুণ করে w এবং b-কে আপডেট করা হয়, যাতে তারা আস্তে আস্তে সঠিক মানের দিকে এগোতে পারে:

$$w = w - \alpha \cdot \frac{\partial J}{\partial w}$$
$$b = b - \alpha \cdot \frac{\partial J}{\partial b}$$

$\alpha$  (Alpha): লার্নিং রেট (Learning Rate)। এটি নির্ধারণ করে মডেল কতটা বড় বা ছোট স্টেপ ফেলে গ্লোবাল মিনিমামের দিকে নামবে

### ৫. কস্ট ফাংশন গ্রাফ (Cost Function Graph)

লিনিয়ার রিগ্রেশনের কস্ট ফাংশনের গ্রাফটি দেখতে একটি বাটি বা বোলের (Bowl) মতো হয়, যাকে গাণিতিক ভাষায় Convex Function বলা হয়। গ্রাডিয়েন্ট ডিসেন্ট পাহাড়ের চূড়া থেকে আস্তে আস্তে পা ফেলে বাটির একদম তলানিতে পৌঁছানোর চেষ্টা করে, যেখানে ভুলের পরিমাণ সবচেয়ে কম।



### ব্যাকএন্ডে কীভাবে কাজ করে? (Full Backend Workflow)

ধরা যাক, আমাদের কাছে একটি ডেটাসেট আছে। ব্যাকএন্ডে পুরো প্রসেসটি নিচের ধাপে ধাপে সম্পন্ন হয়:

- Initialization: শুরুতেই কম্পিউটার আন্দাজে  $w = 0$  এবং  $b = 0$  ধরে নেয়
- Forward Pass (Prediction): এই প্রাথমিক  $w$  এবং  $b$  দিয়ে পুরো ডেটাসেটের জন্য  $\hat{y} = wX + b$  সূত্র ব্যবহার করে প্রেডিকশন হিসাব করা হয়
- Compute Cost: প্রেডিকশন বের করার পর আসল মানের সাথে তুলনা করে MSE সূত্রের মাধ্যমে বর্তমান কস্ট বা ভুলের পরিমাণ ( $J$ ) মাপা হয়
- Backward Pass (Gradient Computation): আংশিক অন্তরীকরণের সূত্রের মাধ্যমে বর্তমান ভুলের জন্য  $w$  এবং  $b$  কতটা দায়ী (অর্থাৎ গ্রাডিয়েন্ট  $\partial J/\partial w$  এবং  $\partial J/\partial b$ ) তা ক্যালকুলেট করা হয়
- Update Parameters: এই গ্রাডিয়েন্ট ব্যবহার করে  $w$  এবং  $b$ -এর পুরানো মান থেকে লার্নিং রেট গুণ করে নতুন মান আপডেট করা হয়
- Iteration: এই ২ থেকে ৫ নম্বর ধাপটি লুপের মাধ্যমে হাজার হাজার বার (যেমন 1,500 বার) চলতে থাকে। প্রতিবারে বাট্টির তলানির দিকে নামতে নামতে কস্ট কমতে থাকে এবং একপর্যায়ে স্থির হয়ে যায়

## From Scratch (No Scikit-Learn)

```
1 import numpy as np
2
3 def make_prediction(X, w, b):
4     return w * X + b
5
6 def cost_function(X, y, w, b):
7     n = len(y)
8     y_pred = make_prediction(X, w, b)
9     return (1 / n) * np.sum((y_pred - y) ** 2)
10
11 def calculate_gradient(X, y, w, b):
12     n = len(y)
13     y_pred = make_prediction(X, w, b)
14     dw = (2 / n) * np.sum((y_pred - y) * X)
15     db = (2 / n) * np.sum((y_pred - y))
16     return dw, db
17
18 def gradient_descent(X, y, w, b, lr, epochs):
19     for i in range(epochs):
20         dw, db = calculate_gradient(X, y, w, b)
21         w = w - lr * dw
22         b = b - lr * db
23         if i % 100 == 0:
24             print("epoch:", i, "cost:", cost_function(X, y, w, b))
25     return w, b
26
27 X = np.array([1, 2, 3, 4, 5])
28 y = np.array([2, 4, 6, 8, 10])
29
30 w, b = gradient_descent(X, y, 0, 0, 0.01, 1000)
31
32 print("Final w:", w)
33 print("Final b:", b)
```

## Linear Regression using Scikit Learn

```
1 import numpy as np
2 from sklearn.linear_model import LinearRegression, SGDRegressor
3 from sklearn.preprocessing import StandardScaler
4
5 X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
6 y = np.array([2, 4, 6, 8, 10])
7
8 lr_model = LinearRegression()
9 lr_model.fit(X, y)
10
11 print("Linear Regression")
12 print("w:", lr_model.coef_[0])
13 print("b:", lr_model.intercept_)
14
15 print("Pred:", lr_model.predict(X))
```

## SDG Regression using Scikit Learn

```
18 scaler = StandardScaler()
19 X_scaled = scaler.fit_transform(X)
20
21 sgd_model = SGDRegressor(
22     learning_rate="constant",
23     eta0=0.01,
24     max_iter=1000,
25     tol=1e-3
26 )
27
28 sgd_model.fit(X_scaled, y)
29
30 print("\nSGD Regression")
31 print("w:", sgd_model.coef_[0])
32 print("b:", sgd_model.intercept_[0])
33 print("Pred:", sgd_model.predict(X_scaled))
```

# Multiple Linear Regression

## মাল্টিপল লিনিয়ার রিগ্রেশন (Multiple Linear Regression)

মাল্টিপল লিনিয়ার রিগ্রেশন মূলত সিম্পল লিনিয়ার রিগ্রেশনেরই সম্প্রসারিত রূপ, যেখানে একাধিক ইনপুট ফিচার  $X_1, X_2, \dots, X_m$  ব্যবহার করা হয়।

### ১. মূল সমীকরণ (The Core Formula)

যখন একাধিক ফিচার থাকে, তখন প্রেডিকশন হয়:

$$\hat{y} = w_1X_1 + w_2X_2 + \dots + w_mX_m + b$$

এটিকে ভেক্টরাইজড ফর্মে লেখা হয়:

$$\hat{y} = XW + b$$

যেখানে:

- $X$ : ইনপুট ফিচার ম্যাট্রিক্স (সাইজ  $n \times m$ )
- $W$ : ওয়েট ভেক্টর  $[w_1, w_2, \dots, w_m]^T$
- $b$ : বায়াস (স্কেলার)
- $\hat{y}$ : প্রেডিক্টেড আউটপুট

### ২. কস্ট ফাংশন (Cost Function)

Mean Squared Error (MSE):

$$J(W, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

ভেক্টর ফর্মে:

$$J(W, b) = \frac{1}{n} \|XW + b - y\|^2$$

### ৩. কস্ট ফাংশনের ডেরিভেটিভ (Partial Derivatives)

ওয়েটের জন্য গ্রাডিয়েন্ট:

$$\frac{\partial J}{\partial W} = \frac{2}{n} X^T (XW + b - y)$$

বায়াসের জন্য গ্রাডিয়েন্ট:

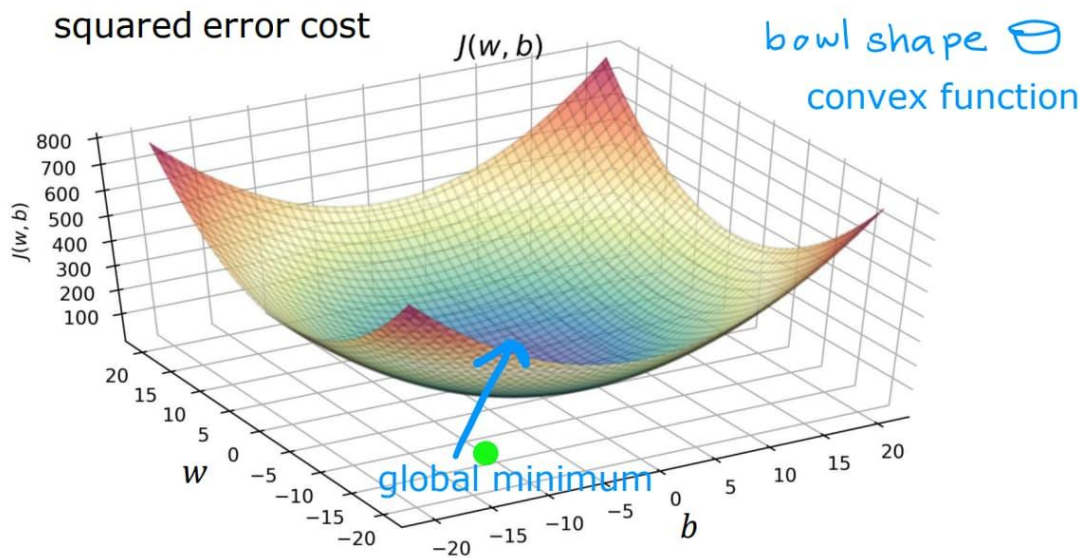
$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum_{i=1}^n (XW + b - y)$$

### ৪. প্যারামিটার আপডেট (Weight & Bias Update)

$$W = W - \alpha \cdot \frac{\partial J}{\partial W}$$
$$b = b - \alpha \cdot \frac{\partial J}{\partial b}$$

### ৫. কস্ট ফাংশন গ্রাফ (Cost Function Graph)

মাল্টিপল ফিচারের ক্ষেত্রে কস্ট ফাংশন একটি **high-dimensional convex surface** তৈরি করে।  
এটি দৃশ্যমানভাবে 2D বা 3D বাটি না হলেও গাণিতিকভাবে একটি মাত্র গ্লোবাল মিনিমাম থাকে।



## ব্যাকএন্ড ওয়ার্কফ্লো (Full Backend Workflow)

- Initialization:  $W = 0, b = 0$
- Forward Pass:

$$\hat{y} = XW + b$$

- Compute Cost: MSE দিয়ে error হিসাব
- Backward Pass:

$$X^T \cdot error$$

- Update Parameters:  $W$  এবং  $b$  আপডেট
- Convergence: লস মিনিমাম না হওয়া পর্যন্ত লুপ

Python Code

```
1 import numpy as np
2
3 X = np.array([
4     [1.0, 2100.0],
5     [2.0, 1500.0],
6     [3.0, 2800.0],
7     [4.0, 1200.0],
8     [5.0, 3200.0]
9 ])
10
11 y = np.array([45.0, 35.0, 60.0, 28.0, 70.0])
12
13 def predict(X, W, b):
14     return np.dot(X, W) + b
15
16 def compute_cost(X, y, W, b):
17     n = len(X)
18     return (1/n) * np.sum((predict(X, W, b) - y) ** 2)
19
20 def compute_gradient(X, y, W, b):
21     n = len(X)
22     error = predict(X, W, b) - y
23     dW = (2/n) * np.dot(X.T, error)
24     db = (2/n) * np.sum(error)
25     return dW, db
```

```

27- def gradient_descent(X, y, W, b, alpha, iters):
28-     for i in range(iters):
29-         dw, db = compute_gradient(X, y, W, b)
30-
31-         W = W - alpha * dw
32-         b = b - alpha * db
33-
34-     return W, b
35-
36- X_mean = np.mean(X, axis=0)
37- X_std = np.std(X, axis=0)
38- X_scaled = (X - X_mean) / X_std
39-
40- W, b = gradient_descent(X_scaled, y, np.zeros(X.shape[1]), 0.0, 0.01, 50000)
41-
42- print(W, b)

```

Using Scikit Learn

```

1  from sklearn.linear_model import LinearRegression, SGDRegressor
2  from sklearn.preprocessing import StandardScaler
3
4  lr = LinearRegression()
5  lr.fit(X, y)
6
7  print(lr.coef_, lr.intercept_)
8
9  scaler = StandardScaler()
10 X_scaled = scaler.fit_transform(X)
11
12 sgd = SGDRegressor(max_iter=10000, eta0=0.01, random_state=42)
13 sgd.fit(X_scaled, y)
14
15 print(sgd.coef_)
16 print(sgd.predict(X_scaled[:1]))

```

লিনিয়ার রিগ্রেশনের ক্ষেত্রে StandardScaler ব্যবহার না করলেও সাধারণত সমস্যা হয় না, কারণ LinearRegression মডেলটি একটি ডাইরেক্ট ম্যাথমেটিক্যাল সলিউশন (OLS) ব্যবহার করে একবারেই ওজন (w) এবং বায়াস (b) এর সঠিক মান বের করে ফেলে। এই পদ্ধতিতে কোনো ধাপে ধাপে ইটারেশন বা লার্নিং প্রসেস নেই, বরং সরাসরি একটি সূত্র ব্যবহার করা হয়:  $(X^T X)^{-1} X^T y$ । ফলে ফিচারের মান বড় বা ছোট যাই হোক না কেন, এটি গ্লোবাল মিনিমাম সঠিকভাবে বের করতে পারে এবং স্কেলিং না করলেও এর আউটপুট সাধারণত পরিবর্তিত হয় না।

অন্যদিকে SGD (Stochastic Gradient Descent) সম্পূর্ণ ভিন্নভাবে কাজ করে। এটি ধাপে ধাপে ইটারেশনের মাধ্যমে ধীরে ধীরে সঠিক মানের দিকে এগিয়ে যায়। এখানে ফিচারের স্কেল বড় হলে গ্রাডিয়েন্ট হিসাবের সময় অত্যন্ত বড় মান তৈরি হতে পারে, যার ফলে ওয়েট হঠাৎ অনেক বেশি বেড়ে গিয়ে মডেল অস্থিতিশীল হয়ে যেতে পারে বা কস্ট ফাংশন ডাইভার্জ করে NaN হয়ে যেতে পারে। এছাড়া স্কেল না করা ডেটায় কস্ট ফাংশনের সারফেস অসম বা লম্বাটে হয়ে যায়, ফলে গ্রাডিয়েন্ট ডিসেন্ট সোজা পথে নামতে না পেয়ে জিগজ্যাগ পথে ধীরে ধীরে কনভার্জ করে। এই কারণে SGD ব্যবহার করার আগে StandardScaler দিয়ে ফিচারগুলোকে একই স্কেলে আনা একদম গুরুত্বপূর্ণ, যাতে ট্রেনিং স্থিতিশীল, দ্রুত এবং নির্ভুল হয়।

## লিনিয়ার রিগ্রেশন (LinearRegression) এবং SGD কীভাবে কাজ করে

### ১. Linear Regression (OLS) কীভাবে কাজ করে

লিনিয়ার রিগ্রেশন একটি **direct mathematical solution** ব্যবহার করে। এখানে মডেল কোনো লুপ চালিয়ে শেখে না। বরং পুরো ডেটাসেট একবারে দেখে একটি নির্দিষ্ট গাণিতিক সূত্র ব্যবহার করে সরাসরি সেরা ওজন (w) এবং বায়াস (b) বের করে ফেলে।

এই সূত্রটি হলো:

$$\theta = (X^T X)^{-1} X^T y$$

এখানে মডেল আসলে এমনভাবে কাজ করে যেন সে বলছে:

“আমি পুরো ডেটা একসাথে দেখে সবচেয়ে সঠিক রেখাটা একবারেই হিসাব করে ফেলছি।”

- ◆ তাই এটি দ্রুত
- ◆ তাই এটি স্থিতিশীল
- ◆ এবং ছোট-মাঝারি ডেটাসেটের জন্য খুব ভালো

### ২. SGD (Stochastic Gradient Descent) কীভাবে কাজ করে

SGD সম্পূর্ণ ভিন্নভাবে কাজ করে। এটি কোনো সরাসরি সূত্র ব্যবহার করে না।

বরং এটি:

- প্রথমে w এবং b র্যান্ডমভাবে নেয়
- তারপর প্রতিটি ডেটা পয়েন্ট বা ছোট ব্যাচ দেখে
- ধাপে ধাপে ভুল কমানোর চেষ্টা করে
- প্রতিবার একটু একটু করে w এবং b আপডেট করে

এটা অনেকটা এমন:

“আমি অন্ধভাবে ধাপে ধাপে হাঁটছি, ভুল হলে দিক ঠিক করছি।”

এই আপডেট হয় এইভাবে:

$$w = w - \alpha \frac{\partial J}{\partial w}$$

### ৩. কখন কোনটা ব্যবহার করা উচিত

**Linear Regression ব্যবহার করা ভালো যখন:**

- ডেটাসেট ছোট বা মাঝারি (few thousand rows পর্যন্ত)
- তুমি দ্রুত এবং exact solution চাও
- কম্পিউটেশনাল রিসোর্স সমস্যা না
- ব্যাচ ট্রেনিং দরকার নেই

উদাহরণ:

- house price prediction
- simple business forecasting
- academic projects

**SGD ব্যবহার করা ভালো যখন:**

- ডেটাসেট অনেক বড় (millions of rows)
- online learning দরকার (data আসতে থাকে নতুন নতুন করে)
- memory সীমিত
- deep learning বা large-scale ML system

উদাহরণ:

- recommendation system
- real-time prediction system
- large-scale industrial ML

## OLS (Ordinary Least Squares) কীভাবে কাজ করে

OLS হলো Linear Regression-এর একটি **closed-form solution** যেখানে মডেল কোনো iteration ছাড়াই সরাসরি best parameters (W, b) বের করে ফেলে।

### ১. Model Equation

Linear Regression-এর মূল সমীকরণ:

$$\hat{y} = XW + b$$

### ২. Cost Function (MSE)

OLS যেটা মিনিমাইজ করে:

$$J(W, b) = \frac{1}{n} \|XW + b - y\|^2$$

এটি মূলত prediction error-এর square mean।

### ৩. Goal of OLS

OLS-এর লক্ষ্য:

$$\min_{W, b} J(W, b)$$

অর্থাৎ এমন W এবং b খুঁজে বের করা যাতে error সবচেয়ে কম হয়।

### ৪. Bias Handling (Important Step)

গণনা সহজ করার জন্য b-কে X-এর সাথে যোগ করা হয়:

$$X' = [X \ 1]$$

এখন সমীকরণ হয়:

$$\hat{y} = X'\theta$$

যেখানে:

- $\theta$  = সব parameters (W এবং b একসাথে)

## ৫. Closed-form Solution (Main Formula)

OLS সরাসরি এই formula ব্যবহার করে solution বের করে:

$$\theta = (X^T X)^{-1} X^T y$$

## ৬. Step-by-step Mechanism

### Step 1: Design Matrix তৈরি

$$X \rightarrow X'$$

### Step 2: Matrix Multiplication

$$X^T X$$

### Step 3: Inverse নেওয়া

$$(X^T X)^{-1}$$

### Step 4: Final multiplication

$$\theta = (X^T X)^{-1} X^T y$$

## OLS (Ordinary Least Squares) — Example

ধরা যাক আমাদের কাছে একটি ছোট dataset আছে:

| X (Study Hours) | y (Marks) |
|-----------------|-----------|
| 1               | 2         |
| 2               | 4         |
| 3               | 6         |

আমরা চাই relationship বের করতে:

$$\hat{y} = wX + b$$

## ১. Design Matrix তৈরি (Bias সহ)

OLS-এ bias যোগ করার জন্য X-কে expand করা হয়:

$$X = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix}$$

এখানে:

- প্রথম column = feature (X)
- দ্বিতীয় column = bias (1)

## ২. Target Vector

$$y = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

## ৩. OLS Formula

$$\theta = (X^T X)^{-1} X^T y$$

এখানে:

$$\theta = \begin{bmatrix} w \\ b \end{bmatrix}$$

## ৪. Step-by-step Calculation

### Step 1: Transpose

$$X^T = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix}$$

### Step 2: Multiply $X^T X$

$$X^T X = \begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix}$$

### Step 3: Inverse

$$\left( (X^T X)^{-1} = \begin{bmatrix} 1.5 & -3 \\ -3 & 7 \end{bmatrix} \right)$$

### Step 4: Multiply with $X^T y$

$$X^T y = \begin{bmatrix} 28 \\ 12 \end{bmatrix}$$

## Final Result

$$\theta = \begin{bmatrix} w \\ b \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

## ৫. Final Model

$$\hat{y} = 2X + 0$$

### LinearRegression (sklearn.linear\_model.LinearRegression)

এটি OLS (Ordinary Least Squares) এর closed-form solution ব্যবহার করে, তাই এখানে regularization বা learning-related কোনো parameter নেই। টিউন করার মতো parameter খুবই কম।

| Parameter                              | অর্থ                                                        | কখন গুরুত্বপূর্ণ                                                                                                            |
|----------------------------------------|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>fit_intercept</b><br>(default=True) | Bias term (b) যুক্ত করবে কিনা                               | যদি তোমার ডেটা already mean-centered হয় বা তুমি জানো রেখা origin দিয়ে যাবে, তখন False করতে পারো। সাধারণত True রাখাই উচিত। |
| <b>positive</b><br>(default=False)     | সব coefficient-কে non-negative ( $\geq 0$ ) রাখতে বাধ্য করে | কিছু physical বা economic dataset-এ যেখানে negative weight অর্থহীন, সেখানে কাজে লাগে।                                       |
| <b>n_jobs</b>                          | Parallel processing-এর জন্য কত core ব্যবহার হবে             | Performance/Speed সংক্রান্ত। Model quality-তে কোনো প্রভাব নেই।                                                              |
| <b>copy_X</b><br>(default=True)        | Input X copy করবে কিনা                                      | Memory optimization-এর জন্য। Model performance-এ প্রভাব নেই।                                                                |

| সবচেয়ে গুরুত্বপূর্ণ  | মন্তব্য                              |
|-----------------------|--------------------------------------|
| <b>fit_intercept</b>  | প্রায় সব ক্ষেত্রে True রাখা উচিত    |
| <b>positive</b>       | বিশেষ domain-specific ক্ষেত্রে দরকার |
| <b>n_jobs, copy_X</b> | Performance ও memory সংক্রান্ত       |

### SGDRegressor (sklearn.linear\_model.SGDRegressor)

এটি Gradient Descent ভিত্তিক মডেল। এখানেই আসল hyperparameter tuning হয় এবং model performance-এর উপর সবচেয়ে বেশি প্রভাব ফেলে।

| Parameter                                      | অর্থ                                                   | কখন গুরুত্বপূর্ণ                                                           |
|------------------------------------------------|--------------------------------------------------------|----------------------------------------------------------------------------|
| <b>loss</b><br>(default='squared_error')       | কোন cost function minimize হবে                         | Outlier বেশি থাকলে 'huber' ব্যবহার করলে model বেশি robust হয়।             |
| <b>penalty</b> (default='l2')                  | Regularization-এর ধরন (l1, l2, elasticnet, None)       | Overfitting কমানোর জন্য অত্যন্ত গুরুত্বপূর্ণ।                              |
| <b>alpha</b> (default=0.0001)                  | Regularization strength                                | সবচেয়ে গুরুত্বপূর্ণ parameter। বেশি হলে underfitting, কম হলে overfitting। |
| <b>l1_ratio</b> (default=0.15)                 | ElasticNet-এ L1 ও L2-এর অনুপাত                         | penalty='elasticnet' হলে ব্যবহার হয়।                                      |
| <b>learning_rate</b><br>(default='invscaling') | Learning rate schedule                                 | Training stability নির্ধারণ করে।                                           |
| <b>eta0</b> (default=0.01)                     | Initial learning rate                                  | খুব বড় হলে divergence, খুব ছোট হলে slow convergence।                      |
| <b>max_iter</b> (default=1000)                 | সর্বোচ্চ কত epoch চলবে                                 | কম হলে model converge করার আগেই থেমে যেতে পারে।                            |
| <b>tol</b> (default=1e-3)                      | Improvement threshold                                  | Training কখন থামবে তা নির্ধারণ করে।                                        |
| <b>early_stopping</b><br>(default=False)       | Validation score উন্নতি না হলে training বন্ধ করবে কিনা | Overfitting কমাতে সাহায্য করে।                                             |
| <b>validation_fraction</b><br>(default=0.1)    | Validation set-এর জন্য ডেটার অংশ                       | early_stopping=True হলে গুরুত্বপূর্ণ।                                      |
| <b>shuffle</b> (default=True)                  | প্রতি epoch-এর আগে data shuffle হবে কিনা               | সাধারণত True রাখা উচিত।                                                    |
| <b>random_state</b>                            | Random seed                                            | Reproducibility-এর জন্য।                                                   |
| <b>average</b> (default=False)                 | Weight averaging ব্যবহার করবে কিনা                     | Noisy data-তে stability বাড়াতে পারে।                                      |

|                                   |                                       |                                      |
|-----------------------------------|---------------------------------------|--------------------------------------|
| <b>warm_start (default=False)</b> | আগের training থেকে continue করবে কিনা | Incremental learning-এর জন্য উপকারী। |
|-----------------------------------|---------------------------------------|--------------------------------------|

### SGDRegressor-এর সবচেয়ে গুরুত্বপূর্ণ Parameter

| Priority | Parameter      | কারণ                    |
|----------|----------------|-------------------------|
| ☆☆☆      | alpha          | Regularization strength |
| ☆☆☆      | penalty        | কোন regularization হবে  |
| ☆☆☆      | learning_rate  | Training stability      |
| ☆☆☆      | eta0           | Initial learning rate   |
| ☆☆       | max_iter       | Convergence নিশ্চিত করে |
| ☆☆       | early_stopping | Overfitting কমায়       |

| বিষয়                  | মন্তব্য                                                           |
|------------------------|-------------------------------------------------------------------|
| <b>Feature Scaling</b> | SGDRegressor ব্যবহারের আগে অবশ্যই StandardScaler ব্যবহার করতে হবে |
| <b>Learning Rate</b>   | ভুল learning rate হলে divergence বা poor convergence হতে পারে     |
| <b>Regularization</b>  | alpha ও penalty ঠিকভাবে tune করা অত্যন্ত গুরুত্বপূর্ণ             |

# Logistic Regression

লজিস্টিক রিগ্রেশন মূলত একটি **ক্লাসিফিকেশন অ্যালগরিদম** (Classification Algorithm), যা বাইনারি ক্লাস (যেমন: 0 অথবা 1, হ্যাঁ অথবা না) প্রেডিক্ট করতে ব্যবহৃত হয়। লিনিয়ার রিগ্রেশন যেখানে  $(-\infty$  থেকে  $\infty)$  পর্যন্ত যেকোনো মান দিতে পারে, লজিস্টিক রিগ্রেশন সেখানে আউটপুটকে 0 থেকে 1 এর মধ্যকার একটি **প্রোবাবিলিটি** (Probability) বা সম্ভাবনায় রূপান্তর করে।

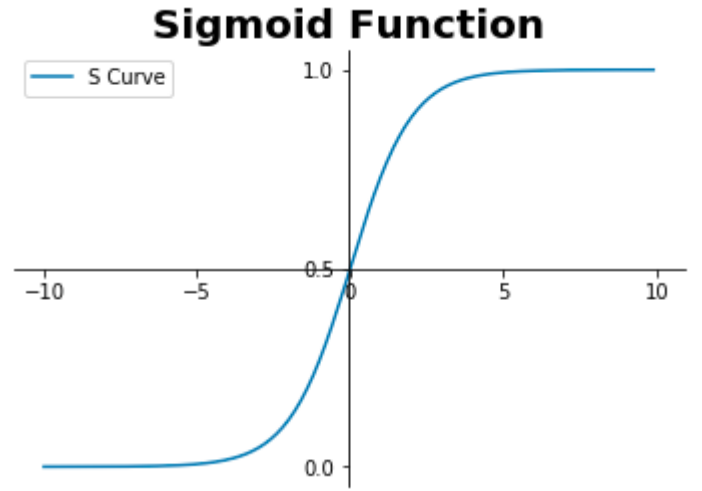
## ১. মূল সমীকরণ ও সিগময়েড ফাংশন (Core Equation & Sigmoid)

প্রথমে linear combination হিসাব করা হয়:

$$z = XW + b$$

এই  $z$ -কে sigmoid function-এর মাধ্যমে probability-তে রূপান্তর করা হয়:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Final prediction:

$$\hat{y} = \sigma(XW + b) = \frac{1}{1 + e^{-(XW+b)}}$$

**Sigmoid-এর গুরুত্বপূর্ণ বৈশিষ্ট্য:**

$$\begin{aligned} z = 0 & \quad \text{তখন } \sigma(z) = 0.5 \\ z \rightarrow +\infty & \quad \text{তখন } \sigma(z) \rightarrow 1 \\ z \rightarrow -\infty & \quad \text{তখন } \sigma(z) \rightarrow 0 \end{aligned}$$

## Odds → Sigmoid Function (Full Derivation)

### ১. Odds কী?

ধরি কোনো event-এর probability হলো  $p$

তাহলে:

$$\text{odds} = \frac{p}{1 - p}$$

মানে: success vs failure ratio

## ২. Log-Odds (Logit)

Odds-এর log নিলে:

$$\log(\text{odds}) = \log\left(\frac{p}{1-p}\right)$$

## ৩. Logistic Regression-এর assumption

$$z = \log\left(\frac{p}{1-p}\right)$$

অর্থাৎ:

- $z = \log\text{-odds}$

## ৪. Log undo করা (Exponential step)

আমরা জানি:

$$z = \log(\text{odds})$$

দুই পাশে exponential নিলে:

$$e^z = \text{odds}$$

## ৫. Odds বসানো

$$\frac{p}{1-p} = e^z$$

## ৬. এখন solve করি probability

$$\begin{aligned}p &= e^z(1-p) \\p &= e^z - pe^z \\p + pe^z &= e^z \\p(1 + e^z) &= e^z\end{aligned}$$

## ৭. Final probability form

$$p = \frac{e^z}{1 + e^z}$$

## ৮. Sigmoid form এ রূপান্তর

উপরের expression কে rearrange করলে:

$$p = \frac{1}{1 + e^{-z}}$$

## ৯. Final Result (Sigmoid Function)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

## ২. কস্ট ফাংশন (Binary Cross-Entropy / Log Loss)

Logistic Regression-এ Mean Squared Error ব্যবহার করা হয় না, কারণ এটি non-convex সমস্যা তৈরি করে।

তাই ব্যবহার করা হয়:

$$J(W, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

### Loss Intuition

যখন  $y = 1$ :

$$\text{Loss} = -\log(\hat{y})$$

- $\hat{y} \approx 1 \rightarrow \text{loss} \approx 0$
- $\hat{y} \approx 0 \rightarrow \text{loss} \rightarrow \infty$

যখন  $y = 0$ :

$$\text{Loss} = -\log(1 - \hat{y})$$

- $\hat{y} \approx 0 \rightarrow \text{loss} \approx 0$
- $\hat{y} \approx 1 \rightarrow \text{loss} \rightarrow \infty$

## ৩. গ্রাডিয়েন্ট (Partial Derivatives)

Log loss + sigmoid combine করলে খুব সুন্দরভাবে gradient simplify হয়।

**Linear part:**

$$z = XW + b$$

$$\hat{y} = \sigma(z)$$

**Weight gradient:**

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

**Bias gradient:**

$$\frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

**Important Note:**

এখানে:

- $\hat{y}$  = sigmoid output
- error =  $\hat{y} - y$

## ৪. প্যারামিটার আপডেট (Gradient Descent)

$$W = W - \alpha \cdot \frac{\partial J}{\partial W}$$

$$b = b - \alpha \cdot \frac{\partial J}{\partial b}$$

## ব্যাকএন্ড ওয়ার্কফ্লো (Full Process)

### Step 1: Initialization

- $W = 0$  বা ছোট random value
- $b = 0$

### Step 2: Linear computation

$$z = XW + b$$

**Step 3: Activation (Sigmoid)**

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

**Step 4: Loss computation**

$$J(W, b)$$

**Step 5: Gradient computation**

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

**Step 6: Update parameters**

$$W, b \text{ update}$$

**Step 7: Prediction rule**

$$\begin{aligned} \hat{y} \geq 0.5 &\Rightarrow 1 \\ \hat{y} < 0.5 &\Rightarrow 0 \end{aligned}$$

## Python Code

```
1 import numpy as np
2
3 X = np.array([
4     [0.5, 1.5],
5     [1.0, 1.0],
6     [1.5, 0.5],
7     [3.0, 3.5],
8     [2.5, 4.5],
9     [4.0, 3.0]
10 ])
11
12 y = np.array([0, 0, 0, 1, 1, 1])
13
14 def sigmoid(z):
15     return 1 / (1 + np.exp(-z))
16
17 def predict_proba(X, W, b):
18     return sigmoid(np.dot(X, W) + b)
19
20 def predict(X, W, b, threshold=0.5):
21     return (predict_proba(X, W, b) >= threshold).astype(int)
22
23 def compute_cost(X, y, W, b):
24     y_pred = predict_proba(X, W, b)
25     return -np.mean(y*np.log(y_pred) + (1-y)*np.log(1-y_pred))
26
27 def compute_gradient(X, y, W, b):
28     error = predict_proba(X, W, b) - y
29     dW = (1/len(X)) * np.dot(X.T, error)
30     db = (1/len(X)) * np.sum(error)
31     return dW, db
32
33 def fit(X, y, lr, iters):
34     W = np.zeros(X.shape[1])
35     b = 0.0
36
37     for i in range(iters):
38         dW, db = compute_gradient(X, y, W, b)
39         W -= lr * dW
40         b -= lr * db
41
42     return W, b
43
44 W, b = fit(X, y, 0.1, 5000)
45
46 print(W, b)
47 print(predict(X, W, b))
```

## Using Scikit Learn

```
1 from sklearn.linear_model import LogisticRegression, SGDClassifier
2 from sklearn.preprocessing import StandardScaler
3
4 lr = LogisticRegression(penalty=None)
5 lr.fit(X, y)
6
7 print(lr.coef_, lr.intercept_)
8 print(lr.predict(X))
9
10 scaler = StandardScaler()
11 X_scaled = scaler.fit_transform(X)
12
13 sgd = SGDClassifier(loss="log_loss", max_iter=5000, eta0=0.1, learning_rate
    ="constant")
14 sgd.fit(X_scaled, y)
15
16 print(sgd.coef_, sgd.intercept_)
17 print(sgd.predict(X_scaled))
```

## LogisticRegression (sklearn.linear\_model.LogisticRegression)

Logistic Regression classification সমস্যার জন্য ব্যবহৃত হয় এবং regularization-এর উপর অনেক বেশি নির্ভরশীল।

| Parameter                           | অর্থ                                           | কখন গুরুত্বপূর্ণ                                    |
|-------------------------------------|------------------------------------------------|-----------------------------------------------------|
| <b>C (default=1.0)</b>              | Regularization strength-এর inverse             | সবচেয়ে গুরুত্বপূর্ণ parameter।                     |
| <b>penalty (default='l2')</b>       | Regularization type (l1, l2, elasticnet, None) | Overfitting control ও feature selection-এর জন্য।    |
| <b>solver (default='lbfgs')</b>     | Optimization algorithm                         | Penalty-এর উপর নির্ভর করে solver নির্বাচন করতে হয়। |
| <b>class_weight (default=None)</b>  | Imbalanced dataset handle করে                  | Imbalanced classification-এ খুব গুরুত্বপূর্ণ।       |
| <b>max_iter (default=100)</b>       | Optimization iteration সংখ্যা                  | Convergence warning এড়াতে বাড়ানো লাগে।            |
| <b>fit_intercept (default=True)</b> | Bias term যুক্ত হবে কিনা                       | সাধারণত True রাখা উচিত।                             |
| <b>l1_ratio</b>                     | ElasticNet-এ L1/L2 ratio                       | শুধুমাত্র ElasticNet-এর জন্য।                       |
| <b>multi_class</b>                  | Multiclass strategy                            | Multiclass classification-এর ক্ষেত্রে গুরুত্বপূর্ণ। |
| <b>tol (default=1e-4)</b>           | Convergence threshold                          | Training stopping criteria নির্ধারণ করে।            |
| <b>random_state</b>                 | Random seed                                    | Consistent result-এর জন্য।                          |
| <b>warm_start (default=False)</b>   | আগের solution থেকে training continue করবে কিনা | Re-training দ্রুত করতে পারে।                        |
| <b>n_jobs</b>                       | Parallel processing                            | Speed বাড়ায়, model quality পরিবর্তন করে না।       |

### Solver & Penalty Compatibility

| Solver    | L1 | L2 | ElasticNet | None |
|-----------|----|----|------------|------|
| lbfgs     | ✗  | ✓  | ✗          | ✓    |
| newton-cg | ✗  | ✓  | ✗          | ✓    |
| sag       | ✗  | ✓  | ✗          | ✓    |
| liblinear | ✓  | ✓  | ✗          | ✗    |
| saga      | ✓  | ✓  | ✓          | ✓    |

### LogisticRegression-এর সবচেয়ে গুরুত্বপূর্ণ Parameter

| Priority | Parameter    | কারণ                        |
|----------|--------------|-----------------------------|
| ★ ★ ★    | C            | Regularization strength     |
| ★ ★ ★    | penalty      | Overfitting control         |
| ★ ★ ★    | solver       | Optimization compatibility  |
| ★ ★ ★    | class_weight | Imbalanced dataset handling |
| ★ ★      | max_iter     | Proper convergence          |
| ★ ★      | l1_ratio     | ElasticNet tuning           |

| বিষয়             | মন্তব্য                                                        |
|-------------------|----------------------------------------------------------------|
| Feature Scaling   | Logistic Regression-এর জন্যও StandardScaler ব্যবহার করা ভালো   |
| Imbalanced Data   | class_weight='balanced' অনেক ক্ষেত্রে performance বাড়ায়      |
| Feature Selection | penalty='l1' coefficient শূন্য করে feature selection করতে পারে |
| Large Dataset     | বড় dataset-এর জন্য saga solver সাধারণত ভালো কাজ করে           |